Aurora Singer

Relatórios do lançamento foguete Aurora Singer



(FONTE: https://pixabay.com/pt/photos/nave-espacial-foguete-decolar-nasa-774/, acesso em: 24/08/2026 às 09:25)

Luke Malaquias Lage, Rafaela Aparecida dos Santos Atanásio, Guilherme de Souza Santos, Yasmin Sampaio Barbieri, Enzo Yugi Kloiwa

Ciência da Computação - FIAP, CCOMP-S

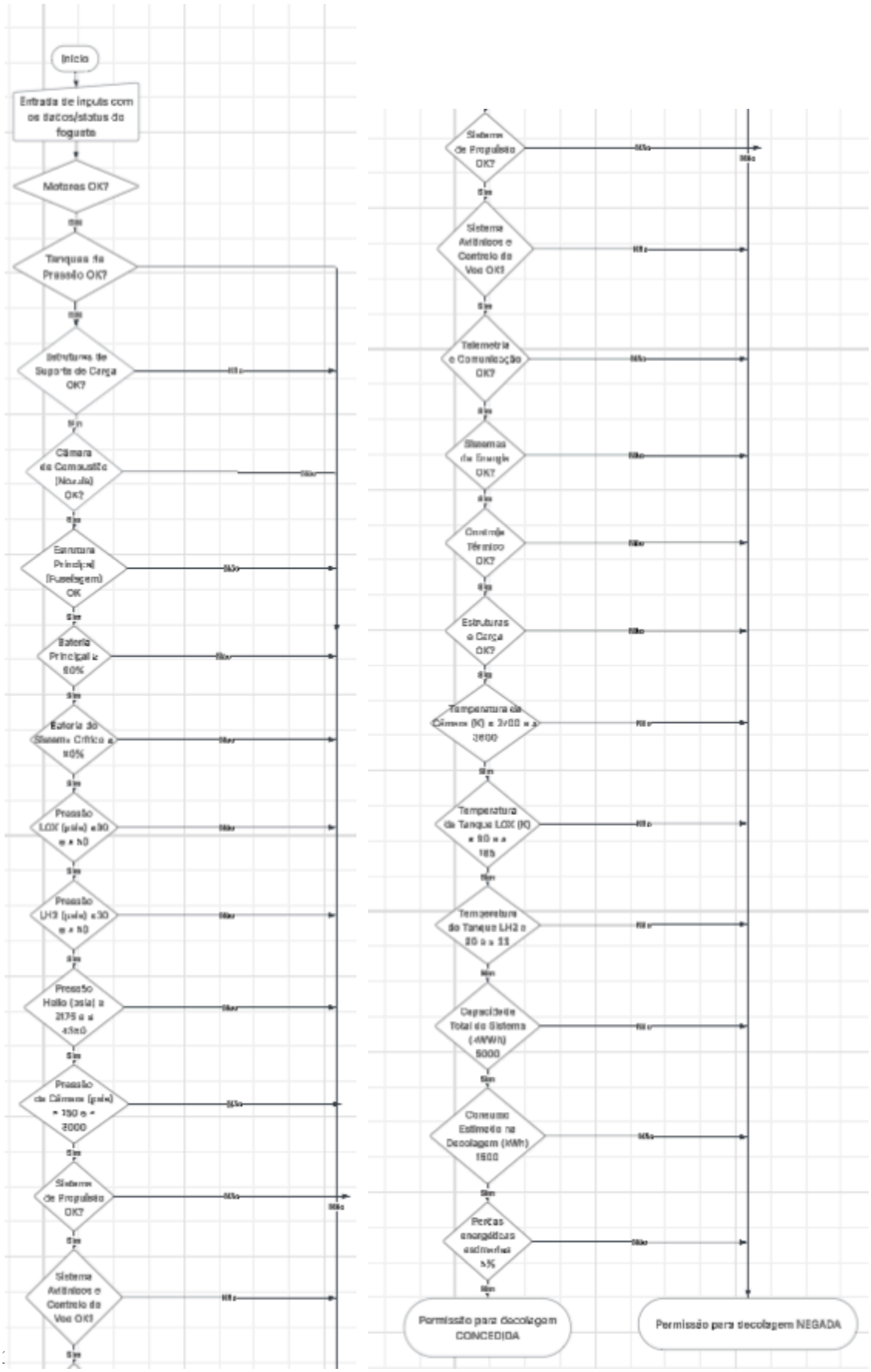
INTRODUÇÃO

O foguete Aurora Singer se baseia em atividades do curso de ciência da computação da faculdade FIAP, neste relatório descrevemos nosso algoritmo, cálculos realizados para entender a telemetria usada para o lançamento, nosso código de permissão e prints deste para avaliação.

ALGORITMO

Nosso algoritmo basicamente funciona permitindo ou recusando o lançamento do foguete Aurora Singer, analisando os parâmetros apresentados e determinando a integridade do foguete, assim determinando se é possível e plausível o lançamento ou não.

Abaixo, o fluxograma algoritmo do lançamento da missão:


Início
Entrada de inputs com os dados/status do foguete
Motores OK?
Não
Tanques de Pressão OK?
Não
Estruturas de Suporte de Carga OK?
Não
Não
Câmara de Combustão (Nozzle) OK?
Não
Sim
Estrutura Principal (Fuselagem) OK
Não
Sim
Bateria Principal ≥ 80%
Não
Sim
Bateria de Sistema Crítico ≥ 90%
Não
Sim
Pressão LOX (psia) ≥ 90 e ≤ 110
Não
Sim
Pressão LH2 (psia) ≥ 30 e ≤ 50
Não
Sim
Pressão Hélio (psia) ≥ 2175 e ≤ 4350
Não
Sim
Pressão da Câmara (psia) ≥ 150 e ≤ 3000
Não
Sim
Sistemas de Propulsão OK?
Não
Não
Sim
Sistema Antifalhas e Controle de Voo OK?
Não
Sim
Telemetria e Comunicação OK?
Não
Sim
Sistemas de Energia OK?
Não
Sim
Controle Térmico OK?
Não
Sim
Estruturas e Carga OK?
Não
Sim
Temperatura da Câmara (K) ≥ 3100 e ≤ 3600
Não
Sim
Temperatura do Tanque LOX (K) ≥ 90 e ≤ 105
Não
Sim
Temperatura do Tanque LH2 ≥ 20 e ≤ 33
Não
Sim
Capacidade Total do Sistema (MWh) ≥ 5000
Não
Sim
Consumo Estimado na Decolagem (MWh) ≤ 1500
Não
Sim
Perdas energéticas estimadas ≤ 5%
Não
Sim
Permissão para decolagem CONCEDIDA
Permissão para decolagem NEGADA

Em um resumo, primeiramente o algoritmo irá ler os dados imputados no sistema pelo operador, para então ler suas informações e entender se:

- A temperatura interna do foguete está entre -10°C e 40°C e a temperatura externa está entre -5° e 40°C;
- Se o gradiente térmico da temperatura está exatamente em 40°C;
- Se o nível de energia está maior ou igual a 70% e o nível de energia do sistema crítico está maior ou igual a 95%;
- Se o nível de oxidante LOX está maior ou igual a 90%;
- Se a pressão RP-1 está entre 200 kPa e 400 kPa e a pressão LOX está entre 300 kPA e 500 kPa;
- Se a integridade estrutural do foguete está completamente intacta;
- E se todos os módulos retornaram status OK.

Caso mesmo apenas um desses parâmetros esteja negativo, toda a missão de lançamento deve ser abortada imediatamente e o foguete não terá permissão de lançamento até os parâmetros retornarem todos completamente OK.

CÓDIGO

```
print("=== AURORA SINGER - CHECKLIST DE DECOLAGEM ===")

# Integridade estrutural (5 componentes)
print("\n-- Integridade estrutural --")
c1 = int(input("motores (1=OK / 0=falha): "))
c2 = int(input("Tanques de pressão (1=OK / 0=falha): "))
c3 = int(input("Estruturas de suporte de carga (1=OK / 0=falha): "))
c4 = int(input("Câmara de combustão(Nozzle) (1=OK / 0=falha): "))
c5 = int(input("Estrutura principal(Fuselagem) (1=OK / 0=falha): "))


# Niveis de energia
print("\n-- Niveis de energia --")
```

```
bateria = float(input("Bateria principal (%): "))

sistema_critico = float(input("Sistema critico (%): "))


# Pressao dos tanques

print("\n-- Pressao dos tanques --")

p_lox = float(input("Pressao LOX (psia): "))

p_lh2 = float(input("Pressao LH2 (psia): "))

p_helio = float(input("Pressao Helio (psi): "))

p_camara = float(input("Pressao da camara (psi): "))


# Status dos modulos criticos (6 modulos)

print("\n-- Status dos modulos criticos --")

m1 = int(input("Sistema de propulsão(1=operacional / 0=falha): "))

m2 = int(input("Aviônicos e controle de voo (1=operacional / 0=falha): "))

m3 = int(input("Telemetria e Comunicação (1=operacional / 0=falha): "))

m4 = int(input("Sistemas de energia (1=operacional / 0=falha): "))

m5 = int(input("Controle Térmico (1=operacional / 0=falha): "))

m6 = int(input("Estrutura e carga (1=operacional / 0=falha): "))


# Temperaturas

print("\n-- Temperaturas --")

t_camara = float(input("Temperatura da camara (K): "))

t_lox = float(input("Temperatura do tanque LOX (K): "))

t_lh2 = float(input("Temperatura do tanque LH2 (K): "))
```

```
# Integridade estrutural
ok_estrutural = (c1 == 1 and c2 == 1 and c3 == 1 and c4 == 1 and c5 == 1)

# Niveis de energia
ok_energia = (bateria >= 70 and sistema_critico >= 70)

# Pressao dos tanques
ok_lox = (30 <= p_lox <= 50)
ok_lh2 = (30 <= p_lh2 <= 50)
ok_helio = (2175 <= p_helio <= 4350)
ok_camara_p = (150 <= p_camara <= 3000)
ok_pressao = (ok_lox and ok_lh2 and ok_helio and ok_camara_p)

# Status dos modulos
ok_modulos = (m1 == 1 and m2 == 1 and m3 == 1 and m4 == 1 and m5 == 1 and
m6 == 1)

# Temperaturas
# Faixas de referencia para os tanques criogenicos (ajustar se o projeto
# definir valores especificos)
ok_temp_camara = (2700 <= t_camara <= 3600)
ok_temp_lox = (90 <= t_lox <= 165)
ok_temp_lh2 = (20 <= t_lh2 <= 33)
```

```
ok_temperaturas = (ok_temp_camara and ok_temp_lox and ok_temp_lh2)


print("\n============================================================")
print("RELATORIO DE VERIFICACAO - AURORA SINGER")
print("============================================================")


if ok_estrutural:
    print("[APROVADO] Integridade estrutural")
else:
    print("[REPROVADO] Integridade estrutural")


if ok_energia:
    print("[APROVADO] Niveis de energia")
else:
    print("[REPROVADO] Niveis de energia")


if ok_pressao:
    print("[APROVADO] Pressao dos tanques")
else:
    print("[REPROVADO] Pressao dos tanques")


if ok_modulos:
    print("[APROVADO] Status dos modulos")
else:
```

```
    print("[REPROVADO] Status dos modulos")


if ok_temperaturas:
    print("[APROVADO] Temperaturas")
else:
    print("[REPROVADO] Temperaturas")

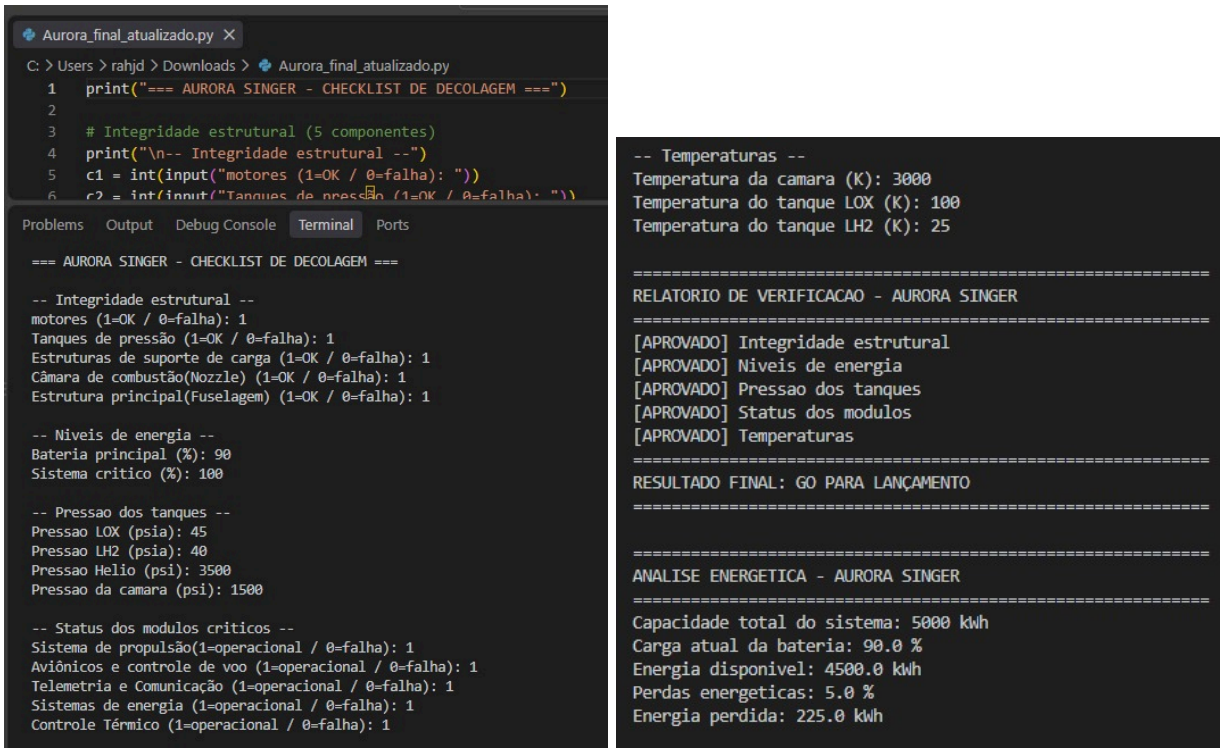

print("============================================================")


if ok_estrutural and ok_energia and ok_pressao and ok_modulos and
ok_temperaturas:
    print("RESULTADO FINAL: GO PARA LANÇAMENTO")
else:
    print("RESULTADO FINAL: NOT-GO - LANÇAMENTO ABORTADO")


print("============================================================")
```

PRINTS DE FUNCIONAMENTO DE CÓDIGO

Abaixo, uma evidência do código funcionando normalmente:



Como visto abaixo, apenas um erro em qualquer um dos parâmetros do sistema aborta a missão.

```
=== AURORA SINGER - CHECKLIST DE DECOLAGEM ===

-- Integridade estrutural --
motores (1=OK / 0=falha): 1
Tanques de pressão (1=OK / 0=falha): 1
Estruturas de suporte de carga (1=OK / 0=falha): 1
Câmara de combustão(Nozzle) (1=OK / 0=falha): 1
Estrutura principal(Fuselagem) (1=OK / 0=falha): 1

-- Niveis de energia --
Bateria principal (%): 87
Sistema critico (%): 97

-- Pressao dos tanques --
Pressao LOX (psia): 50
Pressao LH2 (psia): 50
Pressao Helio (psi): 4350
Pressao da camara (psi): 2900

-- Status dos modulos criticos --
Sistema de propulsão(1=operacional / 0=falha): 1
Aviônicos e controle de voo (1=operacional / 0=falha): 1
Telemetria e Comunicação (1=operacional / 0=falha): 1
Sistemas de energia (1=operacional / 0=falha): 1
Controle Térmico (1=operacional / 0=falha): 1
Estrutura e carga (1=operacional / 0=falha): 1

-- Temperaturas --
Temperatura da camara (K): 2900
Temperatura do tanque LOX (K): 2900
Temperatura do tanque LH2 (K): 130

============================================================
RELATORIO DE VERIFICACAO - AURORA SINGER
============================================================
[APROVADO] Integridade estrutural
[APROVADO] Niveis de energia
[APROVADO] Pressao dos tanques
[APROVADO] Status dos modulos
[REPROVADO] Temperaturas
============================================================
RESULTADO FINAL: NOT-GO - LANÇAMENTO ABORTADO
============================================================
```

ANÁLISE ASSISTIDA POR IA

1. Objetivo da análise

O sistema desenvolvido para a Missão Aurora Singer tem como objetivo analisar dados simulados de telemetria antes da decolagem. O algoritmo verifica diferentes condições operacionais e, a partir dos resultados obtidos, determina se a missão está apta para prosseguir (GO) ou se a decolagem deve ser abortada (NOT-GO).

O programa considera cinco grupos principais de informações: integridade estrutural, níveis de energia, pressão, status dos módulos críticos e temperaturas. Esses dados são fornecidos durante a execução do programa e posteriormente avaliados pelas regras implementadas no algoritmo.

Os valores e limites utilizados são premissas definidas pelo grupo para fins acadêmicos. Portanto, não representam especificações técnicas reais de veículos aeroespaciais.

2. *Lógica de decisão*

O algoritmo utiliza verificações booleanas para analisar cinco grupos principais:

Integridade estrutural;

Níveis de energia;

Pressão;

Status dos módulos críticos;

Temperaturas.

Cada grupo gera um resultado booleano: True, quando todas as condições daquele grupo são atendidas, ou False, quando pelo menos uma condição não é atendida.

A decisão final utiliza o operador lógico and:

```
if ok_estrutural and ok_energia and ok_pressao and ok_modulos and ok_temperaturas:
        print("RESULTADO FINAL: GO PARA LANÇAMENTO")
else:
        print("RESULTADO FINAL: NOT-GO - LANÇAMENTO ABORTADO")
```

Dessa forma, todas as verificações precisam ser verdadeiras simultaneamente para que o lançamento seja autorizado. Se pelo menos uma condição retornar False, o algoritmo apresenta o resultado NOT-GO - LANÇAMENTO ABORTADO. Essa é a lógica implementada no código final do sistema.

Essa estratégia permite representar computacionalmente uma regra simples para a simulação: a decolagem somente é autorizada quando todas as condições previamente estabelecidas pelo grupo são atendidas.

1. *Análise dos subsistemas simulados*

a. *Integridade estrutural*

A primeira verificação realizada pelo programa corresponde à integridade estrutural. O código recebe informações referentes a cinco componentes:

Motores;

Tanques de pressão;

Estruturas de suporte de carga;

Câmara de combustão (Nozzle);

Estrutura principal (Fuselagem).

Cada componente utiliza uma representação binária:

1 — componente em condição adequada;

0 — componente com falha.

Esses dados são recebidos individualmente pelo programa.

A condição utilizada é:

```
ok_estrutural = (
        c1 == 1
        and c2 == 1
        and c3 == 1
        and c4 == 1
        and c5 == 1
)
```

Para que ok_estrutural seja True, os cinco componentes precisam apresentar valor 1. Caso pelo menos um deles apresente valor 0, a integridade estrutural é considerada reprovada.

b. Níveis de energia

O programa também recebe dois valores percentuais relacionados à energia:

Bateria principal;

Sistema crítico.

Essas informações são inseridas como porcentagens durante a execução.

Na simulação, o grupo definiu 70% como valor mínimo para os dois parâmetros:

ok_energia = (bateria >= 70 and sistema_critico >= 70)

Portanto, tanto a bateria principal quanto o sistema crítico precisam apresentar valor igual ou superior a 70%.

Caso um dos dois esteja abaixo desse limite, ok_energia será False e a condição energética será reprovada.

O valor de 70% representa uma premissa adotada pelo grupo exclusivamente para a simulação acadêmica.

c. Pressões

A etapa seguinte verifica quatro valores de pressão informados durante a execução:

Pressão LOX;

Pressão LH2;

Pressão de Hélio;

Pressão da câmara.

O próprio programa solicita esses quatro valores ao usuário.

As condições implementadas são:

ok_lox = (30 <= p_lox <= 50)

ok_lh2 = (30 <= p_lh2 <= 50)

ok_helio = (2175 <= p_helio <= 4350)

ok_camara_p = (150 <= p_camara <= 3000)

ok_pressao = (

    ok_lox

    and ok_lh2

    and ok_helio

    and ok_camara_p

)

Assim, cada valor é comparado com a respectiva faixa estabelecida na simulação.

A variável ok_pressao somente será True quando as quatro verificações de pressão forem verdadeiras ao mesmo tempo. Se pelo menos um valor estiver fora de sua faixa, todo o grupo será reprovado.

Essas faixas são utilizadas como parâmetros simulados definidos para o projeto acadêmico, não como especificações reais de sistemas aeroespaciais.

d. Módulos críticos

O programa verifica também o estado de seis módulos considerados críticos dentro da simulação:

Sistema de propulsão;

Aviônicos e controle de voo;

Telemetria e comunicação;

Sistemas de energia;

Controle térmico;

Estrutura e carga.

Cada módulo é representado por:

1 — operacional;

0 — falha.

Essas são as seis entradas previstas pelo código.

A condição utilizada é:

```
ok_modulos = (
    m1 == 1
    and m2 == 1
    and m3 == 1
    and m4 == 1
```

```
    and m5 == 1
    and m6 == 1
)
```

Todos os seis módulos precisam apresentar valor 1 para que ok_modulos seja True.

Caso qualquer módulo apresente valor 0, a verificação dos módulos críticos será reprovada.

e. Temperaturas

O último grupo de parâmetros analisado corresponde às temperaturas.

O programa solicita três valores:

Temperatura da câmara;

Temperatura do tanque LOX;

Temperatura do tanque LH2.

As condições utilizadas atualmente no código são:

```
ok_temp_camara = (2700 <= t_camara <= 3600)
ok_temp_lox = (90 <= t_lox <= 165)
ok_temp_lh2 = (20 <= t_lh2 <= 33)

ok_temperaturas = (
    ok_temp_camara
    and ok_temp_lox
    and ok_temp_lh2
)
```

Cada temperatura é comparada com a faixa correspondente definida para a simulação.

A variável ok_temperaturas somente será True quando as três verificações forem aprovadas. Caso uma delas esteja fora do intervalo estabelecido, o grupo será reprovado.

Assim como nas verificações anteriores, essas faixas são utilizadas como premissas acadêmicas da simulação.

3. Análise do resultado da execução

Após receber e processar todos os dados, o programa apresenta um relatório contendo o resultado individual de cada um dos cinco grupos.

Para cada grupo, o sistema verifica sua respectiva variável booleana e apresenta APROVADO quando o resultado é True ou REPROVADO quando é False.

A estrutura apresentada pelo programa é:

RELATORIO DE VERIFICACAO - AURORA SINGER

[APROVADO/REPROVADO] Integridade estrutural

[APROVADO/REPROVADO] Niveis de energia

[APROVADO/REPROVADO] Pressao dos tanques

[APROVADO/REPROVADO] Status dos modulos

[APROVADO/REPROVADO] Temperaturas

O código possui verificações individuais para cada um desses resultados.

Após a apresentação dos resultados individuais, ocorre a decisão final.

Se todas as condições forem aprovadas, o programa apresenta:

RESULTADO FINAL: GO PARA LANÇAMENTO

Caso pelo menos uma condição seja reprovada:

RESULTADO FINAL: NOT-GO - LANÇAMENTO ABORTADO

A decisão ocorre porque o algoritmo utiliza o operador and entre os cinco resultados:

ok_estrutural and ok_energia and ok_pressao and ok_modulos and ok_temperaturas

Portanto, mesmo que quatro grupos estejam aprovados, a reprovação de apenas um deles já é suficiente para produzir o resultado NOT-GO.

Esse comportamento demonstra a aplicação prática da lógica booleana e das estruturas condicionais na tomada de decisão realizada pelo programa.

4. Possíveis evoluções do sistema

Embora o sistema atual seja suficiente para demonstrar os conceitos utilizados nesta etapa do projeto, algumas melhorias poderiam ser implementadas futuramente.

Uma possibilidade seria adicionar tratamento de entradas inválidas utilizando estruturas como try/except. Atualmente, as entradas são convertidas diretamente para int ou float. Dessa forma, uma entrada incompatível com o tipo esperado poderia interromper a execução.

Outra possibilidade seria substituir futuramente a entrada manual realizada com input() por dados provenientes de arquivos estruturados, como JSON ou CSV. Isso permitiria simular outra forma de recebimento dos dados de telemetria. Também seria possível organizar as diferentes verificações em funções, separando responsabilidades como:

verificar_integridade()

verificar_energia()

verificar_pressao()

verificar_modulos()

verificar_temperaturas()

Essa organização poderia facilitar a manutenção e a expansão do código em etapas futuras.

Essas possibilidades representam evoluções futuras e não são necessárias para o funcionamento da implementação atual.

5. Conclusão

A análise do sistema de verificação pré-decolagem da Missão Aurora Singer demonstra a aplicação de conceitos fundamentais de programação para processar dados simulados de telemetria e produzir uma decisão.

O programa recebe informações relacionadas à integridade estrutural, energia, pressões, módulos críticos e temperaturas. Em seguida, utiliza operadores relacionais para verificar se os valores atendem às condições estabelecidas na simulação.

As verificações individuais produzem valores booleanos (True ou False) e são posteriormente

combinadas por meio do operador lógico and.

Dessa forma, o resultado GO PARA LANÇAMENTO somente é apresentado quando integridade estrutural, energia, pressão, módulos críticos e temperaturas são aprovados simultaneamente. Caso qualquer uma dessas condições seja reprovada, o sistema apresenta NOT-GO - LANÇAMENTO ABORTADO.

O projeto demonstra, portanto, como a entrada de dados, variáveis, operadores relacionais, lógica booleana e estruturas condicionais podem ser integrados para construir um algoritmo capaz de analisar informações e apoiar uma decisão.

REFLEXÃO CRÍTICA

A exploração espacial, mesmo que muito promissora para o avanço científico e tecnológico, levanta questões éticas profundas sobre responsabilidade, impacto social e sustentabilidade, exigindo que sempre haja uma reflexão crítica sobre quem se beneficia, quem assume os riscos e como preservar o cosmos para as futuras gerações.

Ética e responsabilidade na exploração espacial

A ética espacial envolve dilemas sobre direitos, governança e preservação. O Tratado do Espaço Exterior (1967) declara o espaço como "patrimônio comum da humanidade", mas a crescente comercialização - com empresas privadas minerando asteroides ou oferecendo turismo orbital - desafia esse princípio.

Questões centrais incluem:

- **Quem tem o direito de explorar?** Países, corporações ou toda a humanidade? A ausência de uma governança global efetiva permite que atores com mais recursos dominem a exploração, potencialmente exacerbando desigualdades.
- **Direitos humanos no espaço:** Colonos lunares ou marcianos teriam os mesmos direitos legais e humanos que na Terra? Como garantir justiça social em bases espaciais isoladas?
- **Contaminação biológica:** O risco de levar microrganismos terrestres a outros planetas (contaminação direta) ou trazer patógenos desconhecidos à Terra (contaminação reversa) exige protocolos rigorosos de proteção planetária.

Impacto social e desigualdade

A exploração espacial não ocorre no vácuo social. Enquanto bilhões enfrentam fome, pobreza e crise climática na Terra, investimentos bilionários em missões espaciais - especialmente no turismo orbital - são criticados como símbolos de privilégio e desconexão das prioridades globais. Além disso:

- **Acesso desigual**: Apenas indivíduos extremamente ricos ou cidadãos de poucas nações podem participar diretamente da exploração espacial, reforçando disparidades históricas.
- **Benefícios concentrados**: Avanços tecnológicos derivados do espaço (como satélites de comunicação ou monitoramento climático) nem sempre chegam de forma equitativa às populações mais vulneráveis.

Sustentabilidade tecnológica e ambiental

A sustentabilidade no espaço é um desafio técnico e ético. O acúmulo de lixo espacial - mais de 130 milhões de fragmentos em órbita terrestre - ameaça satélites, estações espaciais e futuras missões, podendo levar ao "efeito Kessler", uma cascata de colisões que tornaria certas órbitas inutilizáveis. Outras preocupações incluem:

- **Emissões de foguetes:** Lançamentos frequentes emitem gases e partículas na alta atmosfera, com impactos ainda pouco compreendidos no clima global.
- **Mineração predatória:** A extração de recursos lunares ou asteroides sem regulamentação pode repetir erros da exploração terrestre, esgotando recursos e degradando ambientes únicos.
- **Tecnologias descartáveis:** Foguetes não reutilizáveis e satélites com vida útil curta aumentam o desperdício e o custo ambiental. A adoção de designs reutilizáveis e missões com plano de desorbitação é essencial.

Caminhos para uma exploração ética e sustentável

Para que a exploração espacial beneficie a humanidade como um todo, é necessário:

- **Governança global inclusiva:** Criar mecanismos multilaterais que garantam transparência, equidade e participação de países em desenvolvimento nas decisões sobre o uso do espaço.
- **Princípios de sustentabilidade**: Adotar práticas de "espaço limpo", como reutilização de foguetes, remoção ativa de detritos e proteção de locais científicos e históricos (ex.: locais de pouso da Apollo na Lua).
- **Ética como guia, não como obstáculo**: A ética não deve frear a inovação, mas orientá-la para fins coletivos, evitando que o espaço se torne mais um domínio de exploração colonial e desigualdade.

Em última análise, a exploração espacial deve ser vista não como uma fuga dos problemas da Terra, mas como uma extensão da responsabilidade humana - exigindo que levemos conosco, para as estrelas, os valores de justiça, solidariedade e cuidado com o meio ambiente.